

PROJECT REPORT ON

AUTOMATED DOCKER CONTAINER DEPLOYMENT ON AWS EC2 USING TERRAFORM AND GITHUB ACTIONS

With Prometheus and Grafana Infrastructure Monitoring

Submitted By:

Ayush Saroha

Bachelor of Technology (B.Tech)

Computer Science & Engineering / Information Technology

Under the Guidance of:

Dr. Chavi Ralhan

DevOps Department & Industry Mentor

Date: May 2026

1. Introduction

Modern cloud engineering demands rapid, secure, and reliable deployment systems. The manual administration of servers leads to inconsistencies, service downtime, and security vulnerabilities. This project implements a fully automated hosting pipeline, migrating away from legacy pipeline servers to a serverless GitOps architecture using GitHub Actions and self-hosted runners. Additionally, infrastructure security and configuration are standardized using Terraform as Infrastructure as Code (IaC), and container performance is monitored in real-time using Prometheus and Grafana.

Key technologies employed include AWS EC2 for compute, Docker for virtualization, Terraform for infrastructure automation, GitHub Actions for CI/CD workflow orchestration, and Prometheus/Grafana for continuous observation and alerts.

2. Profile of the Problem / Problem Statement

Modern development teams face three primary operational challenges:

- **High Maintenance Overhead:** Managing dedicated CI/CD servers (such as Jenkins) introduces significant administrative burdens, including operating system upgrades, security patches, plugin compatibility issues, and continuous running costs.
- **Environment Drift:** Discrepancies between local engineering workstations and public cloud hosting platforms often cause deployment failures.
- **Observability Gap:** Operating web servers without localized CPU, memory, traffic, and connection metrics makes diagnosing issues reactive rather than proactive.

Objective: Design and provision a unified hosting infrastructure on AWS that eliminates the need for separate CI/CD server maintenance, packages the application as a portable virtual container, automatically triggers builds and zero-downtime hot-swaps upon Git pushes, and provides an integrated monitoring dashboard for health checks and resource utilization.

3. Existing System

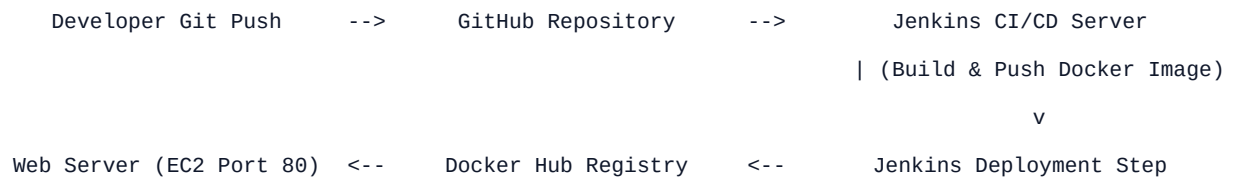
3.1 Introduction

The legacy architecture relied on a dedicated Jenkins master server hosted on an independent AWS virtual machine. Jenkins polled the GitHub repository for changes or received a webhook to trigger builds.

3.2 Existing Software Configuration

- **CI/CD Engine:** Jenkins Server (Running inside VM, maintaining credentials, ssh keys, and historical builds locally).
- **Virtualization:** Docker Engine on EC2.
- **Provisioning:** Manual provisioning or primitive shell scripts.

3.3 Data Flow Diagram (DFD) of Legacy System



3.4 What's New in the Developed System

We have entirely deprecated the Jenkins master server. The new system integrates directly with GitHub Actions via a **self-hosted runner** registered on the target EC2 instance. This eliminates the secondary virtual machine overhead, centralizes secrets management within GitHub, and uses the host environment directly to run tasks. Furthermore, we introduced automated Terraform state syncs and an integrated Prometheus/Grafana stack inside the host virtual network.

4. Problem Analysis

4.1 Product Definition

The product is a containerized web server hosting environment running Nginx on AWS, managed automatically by GitHub Actions runner, and monitored in real-time via Prometheus, Grafana, Node Exporter, and Nginx Exporter.

4.2 Feasibility Analysis

• **Technical Feasibility:** Highly feasible. Leveraging Docker, Ubuntu 22.04 LTS, and official GitHub runner agents makes the system stable and easily replicable. • **Economic Feasibility:** Extremely high. Decommissioning the Jenkins VM saves recurring compute and memory costs. Using GitHub's free-tier Actions runner orchestration further minimizes operational costs. • **Operational Feasibility:** Very high. Developers use standard Git command line interfaces (`git push`), requiring no new software installations or workflow changes.

4.3 Project Plan

- **Phase 1:** Infrastructure definition and Terraform mapping (complete).
- **Phase 2:** Containerization and Nginx health checks configuration (complete).
- **Phase 3:** Self-hosted runner setup and GitHub Actions migration (complete).
- **Phase 4:** Monitoring stack setup (Prometheus, Grafana, Node/Nginx Exporters) (complete).
- **Phase 5:** Verification, integration testing, and documentation (complete).

5. Software Requirement Analysis

5.1 Introduction

This section documents the specific hardware, software, and cloud dependencies required to build, deploy, and maintain the automated web hosting environment.

5.2 General Description

The system operates in a cloud-hosted client-server environment. The developer pushes configuration updates to the remote GitHub Git repository, which acts as the deployment broker. The target EC2 server executes commands locally through the GitHub runner agent to download the updates and restart the web server.

5.3 Specific Requirements

- **Cloud Provider:** Amazon Web Services (AWS) EC2 Virtual Compute
- **Host Operating System:** Ubuntu 22.04 LTS (Jammy Jellyfish)
- **Infrastructure Tool:** HashiCorp Terraform CLI (v1.5.0+)
- **Virtualization:** Docker Engine (v24.0.0+) and Docker Compose (v2.20.0+)
- **CI/CD Agent:** GitHub Actions runner (v2.317.0)
- **Monitoring Stack:** Prometheus v2.53, Grafana v11.2, Node Exporter v1.8, Nginx Exporter v1.3

6. Design

6.1 System Design

The system architecture consists of a public-facing Nginx container (port 80) and a backend monitoring network. The monitoring network scrapes parameters via local loopbacks on the host gateway, protecting the monitoring dashboards (port 3000/9090) from external exposure using AWS Security Group IP restrictions.

6.2 Design Notations (Container Networking Mapping)

Container Name	Host Port	Internal Port	Role / Target
dynamic-site-container	80	80	Main Web Server (Nginx)
grafana	3000	3000	Metrics Dashboard (Visuals)
prometheus	9090	9090	Time-series database (Metrics store)
nginx-exporter	9113	9113	Nginx Stub Status to Prometheus
node-exporter	Host Net	9100	Host hardware details

6.3 Detailed Design (Deployment Pipeline Pseudo-code)

Algorithm: Continuous Deployment on self-hosted runner

1. TRIGGER on Git push event to main branch
2. CHECKOUT repository files onto host directory
3. BUILD Docker image: `docker build -t dynamic-site:latest .`
4. IF dynamic-site-container is active, THEN:
 - a. STOP container: `docker stop dynamic-site-container`
 - b. REMOVE container: `docker rm dynamic-site-container`
5. START new container: `docker run -d --name dynamic-site-container -p 80:80 dynamic-site:latest`
6. PRUNE unused docker resource layers: `docker image prune -f`

7. Testing

7.1 Functional Testing

Functional testing focused on ensuring that the static resources (HTML, CSS, JavaScript) served by Nginx render properly and function without console errors. We performed HTTP tests using `curl` and validated that HTTP header responses return status `200 OK`.

7.2 Structural Testing

Structural testing validated internal networks and Docker bindings. Specifically, the Nginx Stub Status module was evaluated to ensure `/stub\_status` was accessible *only* locally or by the exporter, avoiding data leaks.

7.3 Levels of Testing

- **Unit Testing:** Verified that the Dockerfile builds successfully under localized caching.
- **Integration Testing:** Verified the transition from container-stop to container-run and validated port availability.
- **System Testing:** Verified the complete lifecycle, starting from a Git commit on the client workstation to the live website update on AWS EC2.

7.4 Testing the Project (Test Cases)

Test Case	Input / Action	Expected Output	Status
TC-1	Git Push to Main	GitHub Runner activates	Passed
TC-2	Docker Container Swap	Old stopped, Port 80 taken by new	Passed
TC-3	Prometheus Scraping	Exporters query metrics successfully	Passed
TC-4	Grafana Load	Metrics render on dashboards	Passed

8. Implementation

8.1 Implementation details

Infrastructure was provisioned using Terraform. The EC2 VM runs Ubuntu 22.04 LTS, configured automatically with Docker using the `user\_data.sh` script. The self-hosted runner was registered inside the `/home/ubuntu/actions-runner` folder and configured to run as a system service.

8.2 Conversion Plan (Jenkins to GitHub Actions)

The legacy conversion plan followed a phased cutover: 1. Verify key pair access between the desktop and the AWS target. 2. Add the GitHub Action workflow definition to the repository. 3. Configure and start the runner locally on the EC2 host. 4. Deprecate the Jenkinsfile by stopping the Jenkins master server.

9. Project Legacy

9.1 Current Status

The project is fully complete and operational. The pipeline is running, and live updates take effect instantly upon Git pushes. Dashboards for CPU/Memory (Node Exporter) and HTTP Traffic (Nginx Exporter) are fully configured in Grafana.

9.2 Remaining Areas of Concern

- **Lack of SSL/TLS:** Traffic is currently served over unencrypted HTTP (port 80). Needs Let's Encrypt / Certbot integration.
- **State Management Backups:** The Terraform state is stored locally and should be migrated to an AWS S3 backend for shared access.

9.3 Lessons Learnt

- **Handling Line Endings:** Experienced line ending conflicts (Windows CRLF vs Linux LF) in bash scripts, causing container errors. Resolved by normalizing script carriage returns.
- **AWS Key Immutability:** Learned that AWS KeyPairs are immutable; replacing an SSH key requires changing its logical name in Terraform to force replacement and update EC2 access records.

10. User Manual / Help Guide

10.1 Accessing the Application

To view the running web application, open a web browser and navigate to:

URL: http://18.216.169.166

10.2 Logging into the Monitoring Dashboards

1. Open Grafana: <http://18.216.169.166:3000>
2. Login with **Username:** admin / **Password:** admin
3. Navigate to **Dashboards > Node Exporter Full** to view CPU, memory, and disk health.

10.3 Pushing Application Updates

To update the site, developers simply edit their local code (e.g. `app/index.html`), commit the changes, and push to GitHub. The EC2 server pulls and updates the container automatically.

11. Source Code & System Configurations

11.1 GitHub Actions Workflow (deploy.yml)

```
name: CI/CD Live Deployment on: push: branches: [ main ] jobs: deploy: runs-on: self-hosted steps: - name: Checkout Source Code uses: actions/checkout@v3 - name: Build Docker Image run: docker build -t dynamic-site:latest . - name: Stop and Remove Existing Container run: | docker stop dynamic-site-container || true docker rm dynamic-site-container || true - name: Run New Container run: docker run -d --name dynamic-site-container -p 80:80 --restart always dynamic-site:latest - name: Cleanup Old Images run: docker image prune -f
```

12. Bibliography

1. Docker Documentation: <https://docs.docker.com>
2. Terraform Documentation: <https://registry.terraform.io>
3. GitHub Actions Self-Hosted Runners: <https://docs.github.com/en/actions>
4. Prometheus & Grafana Configuration Guide: <https://prometheus.io/docs>